

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE CODE:** MLA03

**COURSE NAME:** Reinforcement learning for Environment and Agent

---

### COURSE OUTCOME COVERED

**CO3:** *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

*Assessment Tool Weightage: 30%*

---

Dr DUMPALA PRASANTH

**QUESTIONS: 10**

**Total Marks: 50**

Annexure A

### CODING CHALLENGE

**Duration: 2hrs**

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. **(5 marks)**

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. **(5 marks)**

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as  $\epsilon$ -greedy and softmax approaches in balancing learning efficiency. **(5 marks)**

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance. **(5 marks)**

7. Analyze the influence of the discount factor ( $\gamma$ ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. **(5 marks)**

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. **(5 marks)**

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. **(5 marks)**

10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. **(5 marks)**

---

***Code of Conduct:***

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.

***Signature of the student:***

## RUBRICS FOR CALCULATION

| Criteria              | Weightage | Level 4<br>(Exemplary)                                                            | Level 3<br>(Proficient)                          | Level 2<br>(Developing)                            | Level 1<br>(Beginning)                |
|-----------------------|-----------|-----------------------------------------------------------------------------------|--------------------------------------------------|----------------------------------------------------|---------------------------------------|
| Technical Knowledge   | 25        | Demonstrates strong and accurate subject knowledge with in-depth understanding    | Shows good knowledge with minor gaps             | Basic knowledge with noticeable errors             | Lacks fundamental technical knowledge |
| Problem Solving       | 25        | Solves problems logically and applies concepts effectively in real scenarios      | Applies concepts with moderate accuracy          | Limited problem-solving ability; requires guidance | Unable to solve or apply concepts     |
| Analytical Thinking   | 15        | Analyzes questions critically and provides well-structured, logical answers       | Provides reasonable analysis with some structure | Superficial or partially structured responses      | No analytical thinking demonstrated   |
| Communication         | 15        | Communicates clearly, confidently, and professionally with proper terminology     | Communicates well with minor hesitation          | Communication lacks clarity or confidence          | Unable to communicate effectively     |
| Confidence            | 15        | Displays high confidence, positive attitude, and professional behavior throughout | Shows good confidence with minor issues          | Low confidence; inconsistent behavior              | Lacks confidence and professionalism  |
| Response to Questions | 10        | Responds accurately, promptly, and elaborates with relevant examples              | Responds correctly but with limited depth        | Partial responses; needs prompting                 | Unable to respond appropriately       |

## ANSWERS :

### 1. Reinforcement Learning Framework

Reinforcement Learning (RL) is a machine learning approach in which an **agent learns by interacting with an environment**. At each time step, the agent observes the current **state**, selects an **action**, and receives a **reward** from the environment.

The interaction can be represented as:

**State → Action → Reward → New State**

- **Agent:** The learner or decision maker.
- **Environment:** The external system with which the agent interacts.
- **State (S):** Current situation of the environment.
- **Action (A):** Decision taken by the agent.
- **Reward (R):** Feedback received after performing an action.

The agent does not only try to maximize the immediate reward. It aims to maximize the **cumulative future reward**, called the **return**.

where  $\gamma$  (**gamma**) is the discount factor.

**Example:** In a robot navigation problem, the robot's position is the state, moving left/right is an action, and reaching the destination gives a positive reward.

Thus, cumulative reward helps the agent select actions that provide the best **long-term performance** rather than only immediate benefits.

**CODE:**

```
import random
```

```
# States
```

```
states = ["Start", "Middle", "Goal"]
```

```
# Actions
```

```
actions = ["Move Right", "Move Left"]
```

```
# Starting state
```

```
state = "Start"
```

```
total_reward = 0
```

```

print("RL Agent-Environment Interaction\n")

for step in range(3):

    # Agent selects an action
    if state == "Start":
        action = "Move Right"
    elif state == "Middle":
        action = "Move Right"
    else:
        action = "Stay"

    # Environment responds
    if state == "Start" and action == "Move Right":
        next_state = "Middle"
        reward = 1

    elif state == "Middle" and action == "Move Right":
        next_state = "Goal"
        reward = 10

    else:
        next_state = state
        reward = 0

    total_reward += reward

    print("Step:", step + 1)
    print("State:", state)
    print("Action:", action)
    print("Reward:", reward)
    print("Next State:", next_state)
    print()

```

```
state = next_state
```

```
print("Total Cumulative Reward:", total_reward)
```

OUTPUT:

```
... RL Agent-Environment Interaction

Step: 1
State: Start
Action: Move Right
Reward: 1
Next State: Middle

Step: 2
State: Middle
Action: Move Right
Reward: 10
Next State: Goal

Step: 3
State: Goal
Action: Stay
Reward: 0
Next State: Goal

Total Cumulative Reward: 11
```

## 2. Decision-Making as a Markov Decision Process

A **Markov Decision Process (MDP)** is a mathematical framework used to model sequential decision-making problems.

An MDP is represented as:

where:

1. **State space (S):** Set of all possible states.
2. **Action space (A):** Set of actions available to the agent.
3. **Transition probability P:** Probability of moving from one state to another after taking an action.
4. **Reward function R:** Specifies the immediate reward received after taking an action.
5. **Discount factor  $\gamma$ :** Determines the importance of future rewards.

For example, in a self-driving car, the current traffic condition can be the state, acceleration/braking can be actions, the probability of reaching another traffic condition represents transition probability, and safe driving can receive a positive reward.

MDP is useful because it provides a structured representation for solving **sequential decision-making problems**.

CODE:

```
import numpy as np
```

```

# States
states = ["Start", "Middle", "Goal"]

# Actions
actions = ["Right", "Left"]

# Discount factor
gamma = 0.9

# Transition probabilities
P = {
    ("Start", "Right"): {"Middle": 1.0},
    ("Middle", "Right"): {"Goal": 1.0},
    ("Middle", "Left"): {"Start": 1.0}
}

# Rewards
R = {
    ("Start", "Right"): 1,
    ("Middle", "Right"): 10,
    ("Middle", "Left"): -1
}

print("Markov Decision Process\n")

for state in states:
    print("State:", state)

    for action in actions:
        if (state, action) in P:
            print(" Action:", action)
            print(" Transition:", P[(state, action)])
            print(" Reward:", R[(state, action)])

```

```
print("\nDiscount Factor:", gamma)
```

OUTPUT:

```
*** Markov Decision Process

State: Start
Action: Right
Transition: {'Middle': 1.0}
Reward: 1
State: Middle
Action: Right
Transition: {'Goal': 1.0}
Reward: 10
Action: Left
Transition: {'Start': 1.0}
Reward: -1
State: Goal

Discount Factor: 0.9
```

### 3. Policy and Value Functions

A **policy** defines how an agent chooses an action for a given state.

A policy can be deterministic or stochastic.

#### State-Value Function

The state-value function represents the expected cumulative reward when the agent starts from state and follows policy .

#### Action-Value Function

The action-value function represents the expected cumulative reward when the agent takes action in state and then follows policy .

The **Bellman equation** expresses the relationship between the current value and future values:

For Q-values:

Thus, value functions help the agent evaluate **long-term benefits** and select better actions.

CODE:

```
import numpy as np
```

```
# States
```

```
states = ["A", "B", "C"]
```



```

# Rewards
rewards = {
    "A": 2,
    "B": 5,
    "C": 10
}

# Discount factor
gamma = 0.9

# Policy
policy = {
    "A": "Right",
    "B": "Right",
    "C": "Stop"
}

# Value function
V = {}

V["C"] = rewards["C"]
V["B"] = rewards["B"] + gamma * V["C"]
V["A"] = rewards["A"] + gamma * V["B"]

print("Policy:")
for state, action in policy.items():
    print(state, "->", action)

print("\nState Values:")

for state in states:
    print("V(", state, ") =", round(V[state], 2))

# Example Q-value

```

```
Q_A_Right = rewards["A"] + gamma * V["B"]
```

```
print("\nAction Value:")
```

```
print("Q(A, Right) =", round(Q_A_Right, 2))
```

OUTPUT:

```
... Policy:
  A -> Right
  B -> Right
  C -> Stop

State Values:
V( A ) = 14.6
V( B ) = 14.0
V( C ) = 10

Action Value:
Q(A, Right) = 14.6
```

#### 4. Exploration–Exploitation Trade-off

RL agents must balance two important behaviors:

##### Exploration

Trying new actions to discover potentially better rewards.

##### Exploitation

Choosing the action that currently has the highest estimated reward.

If an agent only exploits, it may never discover a better strategy. If it only explores, it may waste time performing poor actions.

##### $\epsilon$ -Greedy Strategy

In  $\epsilon$ -greedy:

- With probability  $\epsilon$ , choose a random action.
- With probability  $1 - \epsilon$ , choose the best-known action.

For example, if  $\epsilon = 0.1$ , the agent explores 10% of the time and exploits 90% of the time.

##### Softmax Strategy

Softmax assigns probabilities to actions based on their Q-values:

where  $\tau$  is the temperature parameter.

| <b><math>\epsilon</math>-Greedy</b>                      | <b>Softmax</b>                |
|----------------------------------------------------------|-------------------------------|
| Random exploration                                       | Probability-based exploration |
| Simple to implement                                      | More gradual action selection |
| Does not consider Q-value differences during exploration | Considers Q-value differences |

Therefore, both methods help achieve a balance between exploration and exploitation.

CODE:

```
import numpy as np
import random

# Q-values of four actions
Q = np.array([2.0, 5.0, 3.0, 1.0])

actions = ["Left", "Right", "Up", "Down"]

# Epsilon
epsilon = 0.2

print("Q-values:")
for a, q in zip(actions, Q):
    print(a, ":", q)

# Epsilon-greedy
if random.random() < epsilon:
    selected_action = random.choice(actions)
    print("\nExploration")
else:
    selected_action = actions[np.argmax(Q)]
    print("\nExploitation")

print("Selected Action:", selected_action)

# Softmax
temperature = 1.0
```

```

exp_values = np.exp(Q / temperature)
probabilities = exp_values / np.sum(exp_values)

print("\nSoftmax Probabilities:")

for action, probability in zip(actions, probabilities):
    print(action, ":", round(probability, 3))

softmax_action = np.random.choice(actions, p=probabilities)

print("\nSoftmax Selected Action:", softmax_action)

```

OUTPUT:

```

... Q-values:
    Left : 2.0
    Right : 5.0
    Up : 3.0
    Down : 1.0

    Exploration
    Selected Action: Right

    Softmax Probabilities:
    Left : 0.041
    Right : 0.831
    Up : 0.112
    Down : 0.015

    Softmax Selected Action: Right

```

## 5. Reward Shaping

**Reward shaping** is the process of modifying or adding rewards to guide an RL agent toward desirable behavior.

Instead of providing a reward only at the final goal, intermediate rewards can be given.

### Example

Consider a robot learning to reach a destination.

Without reward shaping:

- Reaching destination = +100

- Other actions = 0

The agent may take a long time to learn because useful feedback is rare.

With reward shaping:

- Moving closer to destination = +5
- Moving away = -2
- Reaching destination = +100

The agent receives more frequent feedback and can learn faster.

Reward shaping can prevent suboptimal policies by encouraging useful intermediate behaviors. However, poorly designed rewards may cause **reward hacking**, where the agent learns to maximize the artificial reward instead of the actual objective.

Therefore, reward shaping should be designed carefully so that the additional rewards are consistent with the desired goal.

CODE:

```
# Grid environment
```

```
grid = [0, 1, 2, 3, 4]
```

```
start = 0
```

```
goal = 4
```

```
position = start
```

```
total_reward = 0
```

```
print("Reward Shaping Example\n")
```

```
while position < goal:
```

```
    old_position = position
```

```
    # Move toward goal
```

```
    position += 1
```

```
    # Reward for moving closer
```

```
    reward = 2
```

```

# Extra reward for reaching goal
if position == goal:
    reward += 10

total_reward += reward

print("Moved from", old_position, "to", position)
print("Reward:", reward)

print("\nGoal Reached!")
print("Total Reward:", total_reward)

```

OUTPUT:

```

... Reward Shaping Example

Moved from 0 to 1
Reward: 2
Moved from 1 to 2
Reward: 2
Moved from 2 to 3
Reward: 2
Moved from 3 to 4
Reward: 12

Goal Reached!
Total Reward: 18

```

## 9. Q-Learning Update and Convergence

**Q-learning** is an **off-policy, model-free RL algorithm** that learns the optimal action-value function.

The Q-learning update rule is:

where:

- $\alpha$  = learning rate
- $\gamma$  = discount factor
- $R$  = reward
- $s'$  = next state

### Example

Suppose:

Reward:

Next-state maximum Q-value:

Let:

Then:

Thus, the Q-value changes from **10 to 16.5**.

During training, repeated updates gradually improve Q-values. With appropriate learning conditions, sufficient exploration, and suitable learning rates, Q-learning can converge toward the **optimal action-value function** .

CODE:

```
import numpy as np
```

```
import random
```

```
# Number of states and actions
```

```
num_states = 4
```

```
num_actions = 2
```

```
# Q-table
```

```
Q = np.zeros((num_states, num_actions))
```

```
# Parameters
```

```
alpha = 0.5
```

```
gamma = 0.9
```

```
epsilon = 0.2
```

```
# Rewards
```

```
# State 0 -> State 1 -> State 2 -> State 3
```

```
rewards = {
```

```
    (0, 0): 0,
```

```
    (0, 1): 0,
```

```
    (1, 0): 0,
```

```
    (1, 1): 1,
```

```
    (2, 0): 0,
```

```
    (2, 1): 10
```

```

}

print("Initial Q-table:")
print(Q)

# Training
for episode in range(20):

    state = 0

    for step in range(10):

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, num_actions - 1)
        else:
            action = np.argmax(Q[state])

        # Determine next state
        if action == 0:
            next_state = min(state + 1, num_states - 1)
        else:
            next_state = max(state - 1, 0)

        # Reward
        reward = rewards.get((state, action), 0)

        # Q-learning update
        old_q = Q[state, action]

        Q[state, action] = old_q + alpha * (
            reward + gamma * np.max(Q[next_state]) - old_q
        )

```



```

state = next_state

if state == 3:
    break

print("\nFinal Q-table:")
print(np.round(Q, 2))

print("\nLearned Best Actions:")

for state in range(num_states):
    best_action = np.argmax(Q[state])
    print("State", state, "-> Action", best_action)

```

OUTPUT:

```

... Initial Q-table:
[[0. 0.]
 [0. 0.]
 [0. 0.]
 [0. 0.]]

Final Q-table:
[[42.28 33.11]
 [47.06 32.11]
 [ 0.   52.31]
 [ 0.   0.   ]]

Learned Best Actions:
State 0 -> Action 0
State 1 -> Action 0
State 2 -> Action 1
State 3 -> Action 0

```

## 10. On-Policy vs Off-Policy Learning

### On-Policy Learning

An on-policy algorithm learns about the same policy that it uses for taking actions.

**Example:** SARSA.

SARSA update:

where  $a$  is the action actually selected by the current policy.

### Off-Policy Learning

An off-policy algorithm learns an optimal target policy while potentially following a different behavior policy.

**Example:** Q-learning.

| On-Policy                                | Off-Policy                                     |
|------------------------------------------|------------------------------------------------|
| Learns current behavior policy           | Learns a target policy different from behavior |
| Example: SARSA                           | Example: Q-learning                            |
| Generally considers actual actions taken | Uses best estimated future action              |
| Can be safer in some learning situations | Often learns optimal policies efficiently      |
| May be less aggressive                   | Can be more aggressive                         |

**Conclusion:** On-policy methods are useful when the learning behavior itself should be considered, while off-policy methods are useful when learning an optimal policy from experiences generated by another behavior policy.

CODE:

```
import numpy as np
import random

# Parameters
alpha = 0.5
gamma = 0.9

# Q-tables
Q_sarsa = np.zeros((3, 2))
Q_qlearning = np.zeros((3, 2))

# Example experience
state = 0
action = 0
reward = 5
next_state = 1
```

```

# Next action selected by current policy
next_action = 1

# -----
# SARSA
# -----

old_value = Q_sarsa[state, action]

Q_sarsa[state, action] = old_value + alpha * (
    reward + gamma * Q_sarsa[next_state, next_action]
    - old_value
)

# -----
# Q-Learning
# -----

old_value = Q_qlearning[state, action]

Q_qlearning[state, action] = old_value + alpha * (
    reward + gamma * np.max(Q_qlearning[next_state])
    - old_value
)

print("SARSA Q-table:")
print(Q_sarsa)

print("\nQ-Learning Q-table:")
print(Q_qlearning)

print("\nSARSA: On-Policy")
print("Q-Learning: Off-Policy")
OUTPUT:

```

```
... SARSA Q-table:
[[2.5 0. ]
 [0.  0. ]
 [0.  0. ]]

Q-Learning Q-table:
[[2.5 0. ]
 [0.  0. ]
 [0.  0. ]]

SARSA: On-Policy
Q-Learning: Off-Policy
```